

Constraint =**basys3.xdc**

Clock

set_property PACKAGE_PIN W5 [get_ports CLK100MHZ]

set_property IOSTANDARD LVCMOS33 [get_ports CLK100MHZ]

Buttons (Directional Control)

Center Button (Used for Reset)

set_property PACKAGE_PIN U18 [get_ports BTNC]

set_property IOSTANDARD LVCMOS33 [get_ports BTNC]

Up Button

set_property PACKAGE_PIN T18 [get_ports BTNU]

set_property IOSTANDARD LVCMOS33 [get_ports BTNU]

Left Button

set_property PACKAGE_PIN W19 [get_ports BTNL]

set_property IOSTANDARD LVCMOS33 [get_ports BTNL]

Right Button

set_property PACKAGE_PIN T17 [get_ports BTNR]

set_property IOSTANDARD LVCMOS33 [get_ports BTNR]

Down Button

set_property PACKAGE_PIN U17 [get_ports BTND]

set_property IOSTANDARD LVCMOS33 [get_ports BTND]

VGA output

set_property PACKAGE_PIN G19 [get_ports {VGA_R[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {VGA_R[0]}]

set_property PACKAGE_PIN H19 [get_ports {VGA_R[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {VGA_R[1]}]

set_property PACKAGE_PIN J19 [get_ports {VGA_R[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {VGA_R[2]}]

set_property PACKAGE_PIN N19 [get_ports {VGA_R[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {VGA_R[3]}]

```
set_property PACKAGE_PIN J17 [get_ports {VGA_G[0]]}
set_property IOSTANDARD LVCMOS33 [get_ports {VGA_G[0]]}
set_property PACKAGE_PIN H17 [get_ports {VGA_G[1]]}
set_property IOSTANDARD LVCMOS33 [get_ports {VGA_G[1]]}
set_property PACKAGE_PIN G17 [get_ports {VGA_G[2]]}
set_property IOSTANDARD LVCMOS33 [get_ports {VGA_G[2]]}
set_property PACKAGE_PIN D17 [get_ports {VGA_G[3]]}
set_property IOSTANDARD LVCMOS33 [get_ports {VGA_G[3]]}
set_property PACKAGE_PIN N18 [get_ports {VGA_B[0]]}
set_property IOSTANDARD LVCMOS33 [get_ports {VGA_B[0]]}
set_property PACKAGE_PIN L18 [get_ports {VGA_B[1]]}
set_property IOSTANDARD LVCMOS33 [get_ports {VGA_B[1]]}
set_property PACKAGE_PIN K18 [get_ports {VGA_B[2]]}
set_property IOSTANDARD LVCMOS33 [get_ports {VGA_B[2]]}
set_property PACKAGE_PIN J18 [get_ports {VGA_B[3]]}
set_property IOSTANDARD LVCMOS33 [get_ports {VGA_B[3]]}
set_property PACKAGE_PIN P19 [get_ports VGA_HS]
set_property IOSTANDARD LVCMOS33 [get_ports VGA_HS]
set_property PACKAGE_PIN R19 [get_ports VGA_VS]
set_property IOSTANDARD LVCMOS33 [get_ports VGA_VS]
```

7-Segment Display (Ensure these match your CA/CB port widths)

```
set_property PACKAGE_PIN W7 [get_ports {CA[0]]}
set_property IOSTANDARD LVCMOS33 [get_ports {CA[0]]}
set_property PACKAGE_PIN W6 [get_ports {CA[1]]}
set_property IOSTANDARD LVCMOS33 [get_ports {CA[1]]}
# ... (Add remaining segments CA[2-6] and CB as needed)
```

REMOVED: Joystick Analog Inputs (JA1, JA2) are no longer needed.

```

1)clk_div_25mhz.v
// 100 MHz to 25 MHz Clock Divider for Basys 3
// Divides the onboard 100 MHz oscillator to generate
// the 25 MHz pixel clock needed for 640x480 @ 60Hz VGA
module clk_div_25mhz (
    input wire clk_100mhz, // 100 MHz system clock (Basys 3 W5 pin)
    input wire reset,      // Active high reset
    output reg clk_25mhz = 0 // 25 MHz pixel clock output
);

    reg [1:0] count = 2'b00;

    always @(posedge clk_100mhz or posedge reset) begin
        if (reset) begin
            count    <= 2'b00;
            clk_25mhz <= 1'b0;
        end else begin
            if (count == 2'b01) begin
                clk_25mhz <= ~clk_25mhz;
                count    <= 2'b00;
            end else begin
                count <= count + 1'b1;
            end
        end
    end

end

endmodule

```

- food_generator.v

```
module food (  
    input wire clk,  
    input wire reset,  
    output wire [15:0] food_x,  
    output wire [15:0] food_y  
);  
  
    reg [15:0] lfsr = 16'hACE1;  
    wire feedback;  
  
    assign feedback = lfsr[15] ^ lfsr[13] ^ lfsr[12] ^ lfsr[10];  
  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            lfsr <= 16'hACE1;  
        else  
            lfsr <= {lfsr[14:0], feedback};  
        end  
  
    assign food_x = {8'b0, lfsr[7:0]} * 16 + 16'h0020;  
    assign food_y = {8'b0, lfsr[15:8]} * 16 + 16'h0020;  
  
endmodule
```

- game_params.vh

```
// =====

// Snake Game Parameters Header (game_params.vh)
// Basys 3 FPGA | Vivado | Verilog Project
// =====

// Grid dimensions (matches VGA area)
`define GRID_WIDTH    32    // Number of columns (horizontal cells)
`define GRID_HEIGHT   20    // Number of rows (vertical cells)

// Snake settings
`define SNAKE_MAX_LENGTH 100 // Maximum snake segments

// Color definitions (RGB 12-bit: [11:8]=Red, [7:4]=Green, [3:0]=Blue)
`define COLOR_SNAKE    12'h0F0 // Green snake
`define COLOR_FOOD     12'hF00 // Red food
`define COLOR_BG       12'h333 // Dark gray background

// Timing: clock cycles per snake move (controls game speed)
// Adjust this for Basys 3 at 25MHz pixel clock, e.g. ~40 Hz snake
`define MOVE_PERIOD    24'd1000000

// Direction encoding (optional)
`define DIR_UP         2'b00
`define DIR_RIGHT      2'b01
`define DIR_LEFT       2'b10
`define DIR_DOWN       2'b11

// Starting parameters (optional)
//`define SNAKE_START_X  (`GRID_WIDTH/2)
//`define SNAKE_START_Y  (`GRID_HEIGHT/2)
```

```
// =====
// Usage Example:
// `include "game_params.vh"
// Use parameters in Verilog modules: `GRID_WIDTH, etc.
// =====
```

- joystick_input.v

```
module snake_game_top (
    input wire CLK100MHZ,
    input wire BTNC, // Center button for Reset
    input wire BTNU, // Up
    input wire BTND, // Down
    input wire BTNL, // Left
    input wire BTNR, // Right
    output wire [3:0] VGA_R,
    output wire [3:0] VGA_G,
    output wire [3:0] VGA_B,
    output wire VGA_HS,
    output wire VGA_VS,
    output wire [6:0] CA,
    output wire [6:0] CB
);

    wire pixel_clk;
    wire reset;
```

```

wire [1:0] direction;

wire [15:0] food_x, food_y;


assign reset = BTNC; // Basys 3 buttons are active-high


// Connect the new button controller
button_controller control_inst (
    .clk(CLK100MHZ),
    .reset(reset),
    .btnU(BTNU),
    .btnD(BTND),
    .btnL(BTNL),
    .btnR(BTNR),
    .direction(direction)
);


vga_controller vga_inst (
    .pixel_clk(pixel_clk),
    .reset_n(~reset), // VGA usually expects active-low reset_n
    .h_sync(VGA_HS),
    .v_sync(VGA_VS)
);


snake_entity snake_inst (
    .clk(pixel_clk),
    .reset(reset),
    .direction(direction),
    .food_x(food_x),
    .food_y(food_y)
);

```

```
// Note: You will need to add your Score Display logic here  
// to drive CA and CB if that is part of your project.
```

```
endmodule
```

- score_display.v

```
module score (  
    input wire [9:0] point,  
    output reg [6:0] display_r,  
    output reg [6:0] display_l  
);  
  
    reg [6:0] segments [0:9];  
  
    initial begin  
        segments[0] = 7'b1000000; // 0  
        segments[1] = 7'b1111001; // 1  
        segments[2] = 7'b0100100; // 2  
        segments[3] = 7'b0110000; // 3  
        segments[4] = 7'b0011001; // 4  
        segments[5] = 7'b0010010; // 5  
        segments[6] = 7'b0000010; // 6  
        segments[7] = 7'b1111000; // 7  
        segments[8] = 7'b0000000; // 8  
        segments[9] = 7'b0010000; // 9  
    end  
  
    always @(*) begin  
        display_r = segments[point[3:0]];  
        display_l = segments[point[9:4]];
```

end

endmodule

- snake_entity.v

```
module snake_entity (
```

```
    input wire clk,
```

```
    input wire reset,
```

```
    input wire [1:0] direction,
```

```
    input wire [15:0] food_x,
```

```
    input wire [15:0] food_y,
```

```
    output reg [3199:0] snake_out,
```

```
    output reg [9:0] snake_len,
```

```
    output reg game_over
```

```
);
```

```
    reg [15:0] snake_x [0:99];
```

```
    reg [15:0] snake_y [0:99];
```

```
    reg [9:0] length = 10'b1;
```

```
    reg [31:0] speed_counter = 32'b0;
```

```
    localparam SPEED_DIVISOR = 32'h2FAF080;
```

```
    always @(posedge clk or posedge reset) begin
```

```
        if (reset) begin
```

```
            length <= 10'd1;
```

```
            snake_x[0] <= 16'h0100;
```

```
            snake_y[0] <= 16'h0100;
```

```
        end else begin
```

```
            if (speed_counter == SPEED_DIVISOR) begin
```

```
                speed_counter <= 32'b0;
```

```

// Move snake based on direction
case (direction)
    2'b00: snake_y[0] <= snake_y[0] - 16; // Up
    2'b01: snake_x[0] <= snake_x[0] + 16; // Right
    2'b10: snake_x[0] <= snake_x[0] - 16; // Left
    2'b11: snake_y[0] <= snake_y[0] + 16; // Down
endcase

end else begin

    speed_counter <= speed_counter + 1;

end

end

end

always @(*) begin

    snake_len = length;

end

```

endmodule

- snake_game_top.v

```

module snake_game_top (
    input wire CLK100MHZ,

    input wire BTNC,    // Used for Reset

    input wire BTNU,    // Up

    input wire BTND,    // Down

    input wire BTNL,    // Left

    input wire BTNR,    // Right

    output wire [3:0] VGA_R,

    output wire [3:0] VGA_G,

    output wire [3:0] VGA_B,

    output wire VGA_HS,

    output wire VGA_VS,

```

```

output wire [6:0] CA,
output wire [6:0] CB
);

wire pixel_clk;
wire reset;
reg [1:0] direction; // Changed to reg so we can update it logic
wire [15:0] food_x, food_y;

// Internal wires for cleaned-up button signals
wire db_U, db_D, db_L, db_R;

assign reset = ~BTNC; // Assuming your logic uses active-low reset

// --- BUTTON DIRECTION LOGIC ---
// 00: Up, 01: Down, 10: Left, 11: Right (Verify these vs snake_entity.v)
always @(posedge CLK100MHZ) begin
    if (reset) begin
        direction <= 2'b11; // Default starting direction (e.g., Right)
    end else begin
        if (db_U) direction <= 2'b00;
        else if (db_D) direction <= 2'b01;
        else if (db_L) direction <= 2'b10;
        else if (db_R) direction <= 2'b11;
    end
end

// --- INSTANTIATE DEBOUNCERS ---
debouncer dU (.clk(CLK100MHZ), .btn_in(BTNU), .btn_out(db_U));
debouncer dD (.clk(CLK100MHZ), .btn_in(BTND), .btn_out(db_D));
debouncer dL (.clk(CLK100MHZ), .btn_in(BTNL), .btn_out(db_L));

```

```
debouncer dR (.clk(CLK100MHZ), .btn_in(BTNR), .btn_out(db_R));
```

```
vga_controller vga_inst (  
    .pixel_clk(pixel_clk),  
    .reset_n(BTNC),  
    .h_sync(VGA_HS),  
    .v_sync(VGA_VS)  
);
```

```
snake_entity snake_inst (  
    .clk(pixel_clk),  
    .reset(reset),  
    .direction(direction),  
    .food_x(food_x),  
    .food_y(food_y)  
);
```

```
endmodule
```

```
// --- DEBOUNCER MODULE ---
```

```
module debouncer (  
    input clk,  
    input btn_in,  
    output reg btn_out  
);  
    reg [19:0] count;  
    always @(posedge clk) begin  
        if (btn_in) begin  
            if (count < 20'hFFFFFF) count <= count + 1;  
            else btn_out <= 1'b1;  
        end else begin
```

```

        count <= 0;

        btn_out <= 1'b0;

    end

end

endmodule

```

- vga_controller.v

```

module vga_controller (
    input wire pixel_clk,    // 25 MHz for 640x480 @ 60Hz
    input wire reset_n,      // Active low reset
    output reg h_sync,
    output reg v_sync,
    output reg disp_ena,
    output reg [10:0] column,
    output reg [9:0] row
);

```

```

    // 640x480 @ 60Hz VGA standard timing

```

```

    localparam H_VISIBLE = 640;
    localparam H_FRONT  = 16;
    localparam H_SYNC   = 96;
    localparam H_BACK   = 48;
    localparam H_TOTAL  = 800; // FIXED: was 1368

```

```

    localparam V_VISIBLE = 480;
    localparam V_FRONT  = 10;
    localparam V_SYNC   = 2;
    localparam V_BACK   = 33;
    localparam V_TOTAL  = 525; // FIXED: was 768

```

```

    reg [10:0] h_count = 0;
    reg [9:0]  v_count = 0;

```

```

// Pixel counters
always @(posedge pixel_clk or negedge reset_n) begin
    if (!reset_n) begin
        h_count <= 0;
        v_count <= 0;
    end else begin
        if (h_count == H_TOTAL - 1) begin
            h_count <= 0;
            if (v_count == V_TOTAL - 1)
                v_count <= 0;
            else
                v_count <= v_count + 1;
        end else begin
            h_count <= h_count + 1;
        end
    end
end

// Sync pulses (active LOW for 640x480 standard)
always @(*) begin
    h_sync = ~(h_count >= (H_VISIBLE + H_FRONT) &&
        h_count < (H_VISIBLE + H_FRONT + H_SYNC));
    v_sync = ~(v_count >= (V_VISIBLE + V_FRONT) &&
        v_count < (V_VISIBLE + V_FRONT + V_SYNC));
    disp_ena = (h_count < H_VISIBLE && v_count < V_VISIBLE);
end

// Output pixel coordinates
always @(*) begin
    column = h_count;

```

```
    row = v_count;  
end
```

```
endmodule
```